

User-/Kernel-Schnittstellen

Sichere Datenübergaben zwischen zwei Schutzdomänen

3.09

Eine Schnittstelle ist ein langfristiger Vertrag

01 Semantik

Was bedeutet read, write, ioctl oder ein Attribut?

02 Datenformat

Welche Bytes, Felder, Größen und Versionen sind gültig?

03 Fehlerbild

Welche Rückgaben unterscheiden EOF, Teilerfolg und Fehler?

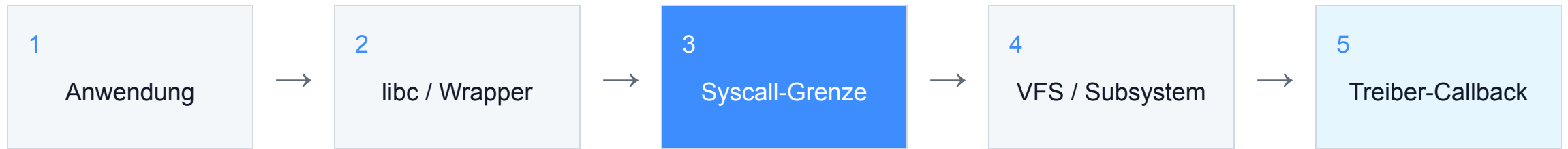
04 Berechtigung

Wer darf lesen, ändern, abonnieren oder mappen?

Konsequenz

Sobald Anwendungen davon abhängen, wird jede Änderung zur Kompatibilitätsentscheidung.

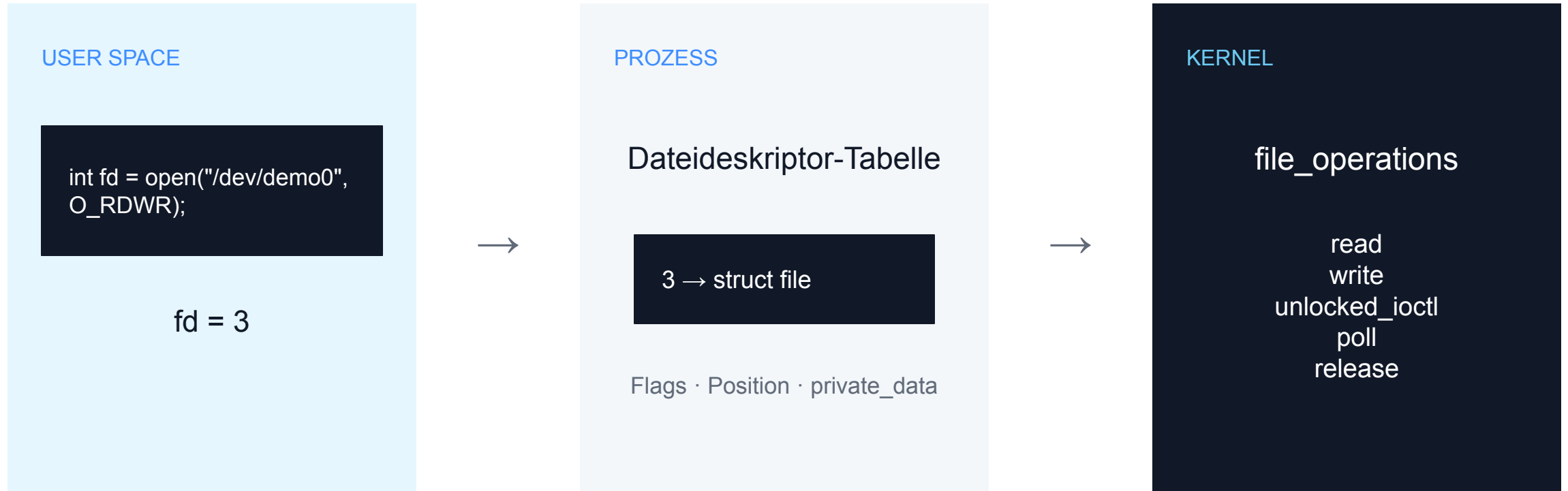
Der Systemaufruf wechselt kontrolliert in den Kernel



`read(fd, buf, count)`

Der Callback läuft typischerweise im Prozesskontext des aufrufenden Tasks und darf bei passenden Bedingungen schlafen.

Dateideskriptoren trennen Handle und Treiber



Mehrere Deskriptoren können getrennten Zustand auf dieselbe Kernelimplementierung beziehen.

Nicht jede Semantik gehört in dieselbe Schnittstelle

Interface	Stärke	Typischer Einsatz
Character Device	fd-bezogen · read/write/ioctl	Gerätedaten und Operationen
sysfs	Attribute von Kernelobjekten	Status und einfache Konfiguration
procfs	System- und Prozessinformation	Beobachtung des laufenden Systems
debugfs	frei und entwicklungsnahe	Diagnose ohne stabile ABI-Garantie
Netlink	strukturierte Nachrichten und Events	Konfiguration, Abfragen, Notifications

Die Schnittstelle folgt dem Kommunikationsmuster – nicht der bequemsten Implementierung.

Die Semantik entscheidet über den Kanal

Stream oder Bytefolge

Character Device

read / write / poll

Einfaches Objektattribut

sysfs

show / store

Entwicklungsdiagnose

debugfs

keine stabile ABI

Strukturierte Events

Netlink

unicast / multicast

Komplexer Befehl pro fd

ioctl

klar versionierte UAPI

Prüffrage: Braucht der Nutzer einen Datenstrom, ein Attribut, einen Befehl oder ein Ereignis?

VFS übersetzt Dateioperationen in Callbacks

```
static const struct file_operations demo_fops = {  
    .owner          = THIS_MODULE,  
    .open           = demo_open,  
    .read           = demo_read,  
    .write          = demo_write,  
    .unlocked_ioctl = demo_ioctl,  
    .poll           = demo_poll,  
    .release        = demo_release,  
};
```

open

Instanz-Zustand vorbereiten

read / write

Daten übertragen

ioctl

strukturierte Befehle

poll

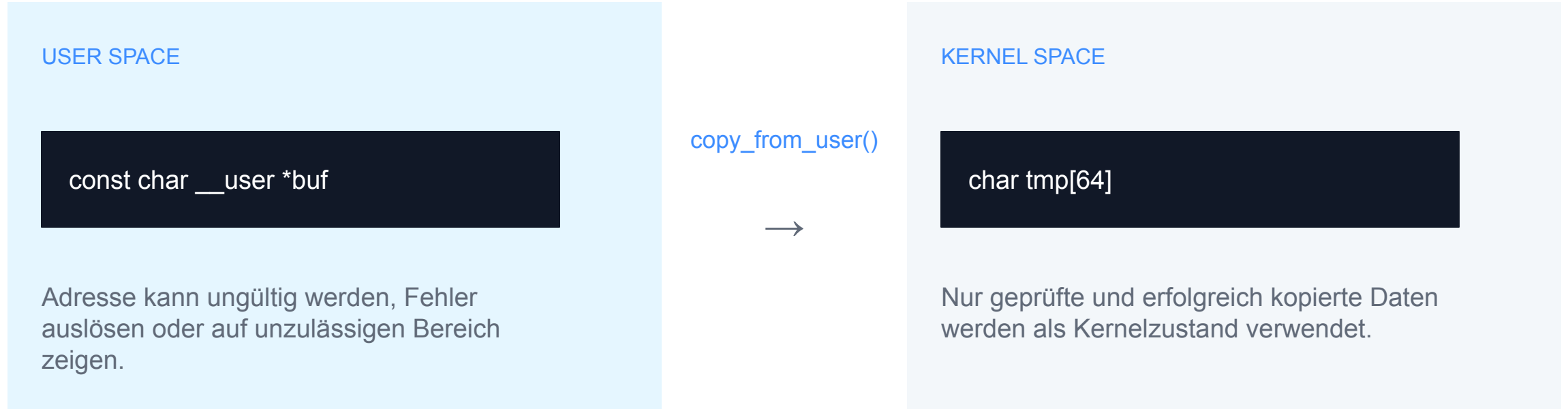
Bereitschaft melden

release

Open-Zustand freigeben

Nur implementierte Operationen anbieten; jede erhält eine klare Rückgabe und einen Lifecycle.

User-Pointer dürfen nicht direkt dereferenziert werden



`__user` dokumentiert den Adressraum; Sparse kann falsche Pointerverwendung statisch sichtbar machen.

copy_from_user importiert Daten in einen Kernelpuffer

```
static ssize_t demo_write(struct file *file,
                        const char __user *buf,
                        size_t count, loff_t *ppos)
{
    char tmp[64];
    size_t len = min(count, sizeof(tmp) - 1);

    if (copy_from_user(tmp, buf, len))
        return -EFAULT;

    tmp[len] = '\0';
    apply_command(tmp);
    return len;
}
```

01 Länge begrenzen

Nie mehr als die Zielkapazität anfordern.

02 Rückgabe prüfen

Nicht kopierte Bytes bedeuten Fehler oder Teilerfolg.

03 Erst danach nutzen

Kernelzustand nur aus vollständig akzeptierten Daten ändern.

copy_to_user exportiert nur vollständig initialisierte Daten

```
static ssize_t demo_read(struct file *file, char __user *buf, size_t count, loff_t *ppos)
{
    size_t avail = strlen(state, sizeof(state));
    size_t len;

    if (*ppos >= avail)
        return 0;

    len = min(count, avail - *ppos);
    if (copy_to_user(buf, state + *ppos, len))
        return -EFAULT;

    *ppos += len;
    return len;
}
```

Offset nur nach erfolgreicher Kopie erhöhen.

get_user und put_user passen zu einzelnen Skalaren

EINLESEN

```
u32 value;  
  
if (get_user(value, user_value))  
    return -EFAULT;
```

AUSGEBEN

```
u32 value = current_value;  
  
if (put_user(value, user_value))  
    return -EFAULT;
```

A Geeignet

char, int und andere einzelne einfache Werte.

B Nicht geeignet

Strukturen, Arrays oder variable Datenblöcke.

Rückgabewerte sind Teil der ABI

Situation	Kernel-Rückgabe	Userspace-Sicht
read liefert Daten	positive Bytezahl	Anzahl verfügbarer Bytes
read am Ende	0	EOF
Useradresse fehlerhaft	-EFAULT	errno = EFAULT
Argument ungültig	-EINVAL	errno = EINVAL
ioctl unbekannt	-ENOTTY	errno = ENOTTY

Positive Werte, 0 und negative Fehlercodes dürfen nicht miteinander verwechselt werden.

UAPI-Strukturen müssen Architekturwechsel überstehen

```
struct demo_config {  
    __u32 size;  
    __u32 flags;  
    __u64 timeout_ns;  
    __u32 reserved[4];  
};
```



Feste Breiten

__u32 / __u64



Explizite Reserve

für spätere Erweiterung



Natürliches Layout

Padding bewusst kontrollieren

Vermeiden

long · Pointer · Bitfields · implizites Padding · uninitialisierte Reservefelder

Userspace-API: Neue Semantik erhält eine neue Command-ID.

Sicherheit beginnt an der Schnittstellengrenze

01 Grenzen

Längen, Indizes, Flags und Wertebereiche prüfen.

02 Berechtigung

Dateimodus, Credentials und Capability passend auswerten.

03 Snapshot

Userdaten einmal kopieren und danach nur die Kernelkopie verwenden.

04 Initialisierung

Keine Paddingbytes oder alten Kernelinhalte zurückgeben.

05 Lifecycle

Objekt beim Zugriff referenzieren und Entfernen synchronisieren.

06 Nebenläufigkeit

Mehrere fds, Threads und Disconnects als Normalfall behandeln.

Die Prüfung endet nicht bei `access_ok`: Bedeutung, Zustand und Berechtigung bleiben Aufgaben des Treibers.

Vier Regeln halten die Schnittstelle beherrschbar

01 Semantik zuerst

Kanal nach Datenstrom, Attribut, Befehl oder Event wählen.

02 Userdaten kopieren

__user markieren, Länge begrenzen, Rückgabe prüfen.

03 ABI dokumentieren

Formate, Fehler, Berechtigungen und Versionierung festlegen.

04 Kompatibel erweitern

Bestehende Bedeutung nicht still verändern.

Nächster Schritt

Character Devices: Gerätenummern, cdev und file_operations praktisch verbinden.